0
Рейтинг
e-legion
Делаем приложения, которыми пользуются миллионы
Подписаться

LeonidBelyakov 7 мар 2021 в 18:48

Получаем результат правильно (Часть 1). Activity Result API

7 мин · 53K

Блог компании e-legion, Разработка мобильных приложений*, Разработка под Android*

Туториал · Из песочницы

registerForActivityResult()
onActivityResult()

Каждый Android-разработчик сталкивался с необходимостью передать данные из одной Activity в другую. Эта тривиальная задача зачастую вынуждает нас писать не самый элегантный код.

Наконец, в 2020 году Google представила решение старой проблемы — Activity Result API. Это мощный инструмент для обмена данными между активностями и запроса runtime permissions.

В данной статье мы разберёмся, как использовать новый API и какими преимуществами он обладает.

Чем плох onActivityResult()?

Роберт Мартин в книге “Чистый код” отмечает важность переиспользования кода — принцип DRY или Don’t repeat yourself, а также призывает проектировать компактные функции, которые выполняют лишь единственную операцию.

Проблема `onActivityResult()` в том, что при его использовании соблюдение подобных рекомендаций становится практически невозможным. Убедимся в этом на примере простого экрана, который запрашивает доступ к камере, делает фото и открывает второй экран — `SecondActivity`. Пусть в `SecondActivity` мы передаём строку, а назад получаем целое значение.

```
class OldActivity : AppCompatActivity(R.layout.a_main) {

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)

       vButtonCamera.setOnClickListener {
           when {
               checkSelfPermission(Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED -> {
                   // доступ к камере разрешен, открываем камеру
                   startActivityForResult(
                       Intent(MediaStore.ACTION_IMAGE_CAPTURE),
                       PHOTO_REQUEST_CODE
                   )
               }
               shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
                   // доступ к камере запрещен, нужно объяснить зачем нам требуется разрешение
               }
               else -> {
                   // доступ к камере запрещен, запрашиваем разрешение
                   requestPermissions(
                       arrayOf(Manifest.permission.CAMERA),
                       PHOTO_PERMISSIONS_REQUEST_CODE
                   )
               }
           }
       }

       vButtonSecondActivity.setOnClickListener {
           val intent = Intent(this, SecondActivity::class.java)
               .putExtra("my_input_key", "What is the answer?")

           startActivityForResult(intent, SECOND_ACTIVITY_REQUEST_CODE)
       }
   }

   override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
       when (requestCode) {
           PHOTO_REQUEST_CODE -> {
               if (resultCode == RESULT_OK && data != null) {
                   val bitmap = data.extras?.get("data") as Bitmap
                   // используем bitmap
               } else {
                   // не удалось получить фото
               }
           }
           SECOND_ACTIVITY_REQUEST_CODE -> {
               if (resultCode == RESULT_OK && data != null) {
                   val result = data.getIntExtra("my_result_extra")
                   // используем result
               } else {
                   // не удалось получить результат
               }
           }
           else -> super.onActivityResult(requestCode, resultCode, data)
       }
   }

   override fun onRequestPermissionsResult(
       requestCode: Int,
       permissions: Array<out String>,
       grantResults: IntArray
   ) {
       if (requestCode == PHOTO_PERMISSIONS_REQUEST_CODE) {
           when {
               grantResults[0] == PackageManager.PERMISSION_GRANTED -> {
                   // доступ к камере разрешен, открываем камеру
                   startActivityForResult(
                       Intent(MediaStore.ACTION_IMAGE_CAPTURE),
                       PHOTO_REQUEST_CODE
                   )
               }
               !shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
                   // доступ к камере запрещен, пользователь поставил галочку Don't ask again.
               }
               else -> {
                   // доступ к камере запрещен, пользователь отклонил запрос
               }
           }
       } else {
           super.onRequestPermissionsResult(requestCode, permissions, grantResults)
       }
   }

   companion object {
       private const val PHOTO_REQUEST_CODE = 1
       private const val PHOTO_PERMISSIONS_REQUEST_CODE = 2
       private const val SECOND_ACTIVITY_REQUEST_CODE = 3
   }
}
```

Очевидно, что метод `onActivityResult()` нарушает принцип единственной ответственности, ведь он отвечает и за обработку результата фотографирования и за получение данных от второй Activity. Да и выглядит этот метод уже довольно запутанно, хоть мы и рассмотрели простой пример и опустили часть деталей.

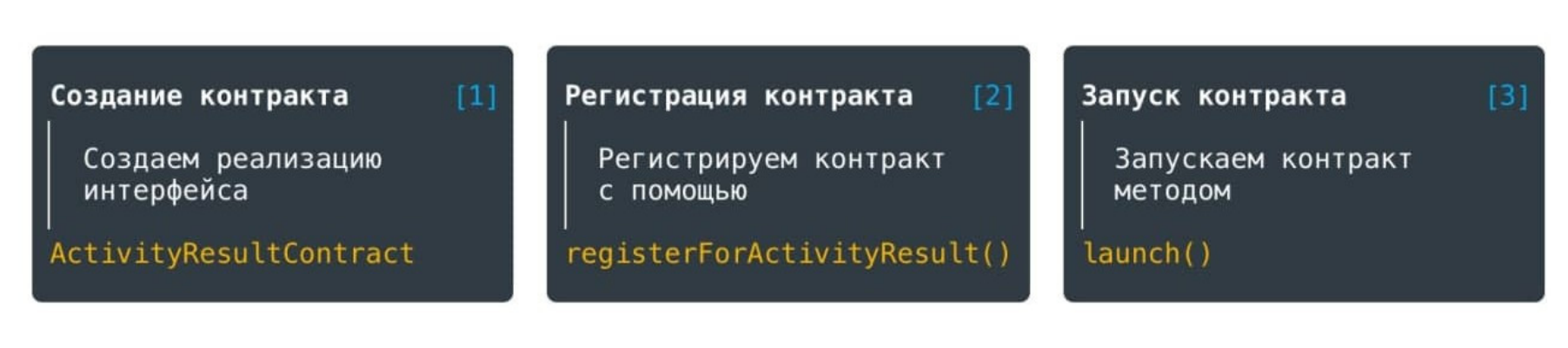
Кроме того, если в приложении появится другой экран со схожей функциональностью, мы не сможем переиспользовать этот код и будем вынуждены его дублировать.

Используем Activity Result API

Новый API доступен начиная с AndroidX Activity `1.2.0-alpha02` и Fragment `1.3.0-alpha02`, поэтому добавим актуальные версии соответствующих зависимостей в build.gradle:

```
implementation 'androidx.activity:activity-ktx:1.3.0-alpha02'
implementation 'androidx.fragment:fragment-ktx:1.3.0'
```

Применение Activity Result состоит из трех шагов:



Создание контракта [1]
Создаем реализацию интерфейса
ActivityResultContract

Регистрация контракта [2]
Регистрируем контракт с помощью
registerForActivityResult()

Запуск контракта [3]
Запускаем контракт методом
launch()

Шаг 1. Создание контракта

Контракт — это класс, реализующий интерфейс `ActivityResultContract<I,O>`. Где `I` определяет тип входных данных, необходимых для запуска Activity, а `O` — тип возвращаемого результата.

Для типовых задач можно воспользоваться реализациями “из коробки”: `PickContact`, `TakePicture`, `RequestPermission` и другими. Полный список доступен тут.

При создании контракта мы обязаны реализовать два его метода:

- `createIntent()` — принимает входные данные и создает интент, который будет в дальнейшем запущен вызовом launch()
- `parseResult()` — отвечает за возврат результата, обработку resultCode и парсинг данных

Ещё один метод — `getSynchronousResult()` — можно переопределить в случае необходимости. Он позволяет сразу же, без запуска Activity, вернуть результат, например, если получены невалидные входные данные. Если подобное поведение не требуется, метод по умолчанию возвращает `null`.

Ниже представлен пример контракта, который принимает строку и запускает SecondActivity, ожидая от неё целое число:

```
class MySecondActivityContract : ActivityResultContract<String, Int?>() {

   override fun createIntent(context: Context, input: String?): Intent {
       return Intent(context, SecondActivity::class.java)
           .putExtra("my_input_key", input)
   }

   override fun parseResult(resultCode: Int, intent: Intent?): Int? = when {
       resultCode != Activity.RESULT_OK -> null
       else -> intent?.getIntExtra("my_result_key", 42)
   }

   override fun getSynchronousResult(context: Context, input: String?): SynchronousResult<Int?>? {
       return if (input.isNullOrEmpty()) SynchronousResult(42) else null
   }
}
```

Шаг 2. Регистрация контракта

Следующий этап — регистрация контракта в активности или фрагменте с помощью вызова `registerForActivityResult()`. В параметры необходимо передать `ActivityResultContract` и `ActivityResultCallback`. Коллбек сработает при получении результата.

```
val activityLauncher = registerForActivityResult(MySecondActivityContract()) { result ->
   // используем result
}
```

Регистрация контракта не запускает новую `Activity`, а лишь возвращает специальный объект `ActivityResultLauncher`, который нам понадобится далее.

Шаг 3. Запуск контракта

Для запуска Activity остаётся вызвать `launch()` на объекте `ActivityResultLauncher`, который мы получили на предыдущем этапе.

```
vButton.setOnClickListener {
   activityLauncher.launch("What is the answer?")
}
```

Важно!

Отметим несколько неочевидных моментов, которые необходимо учитывать:

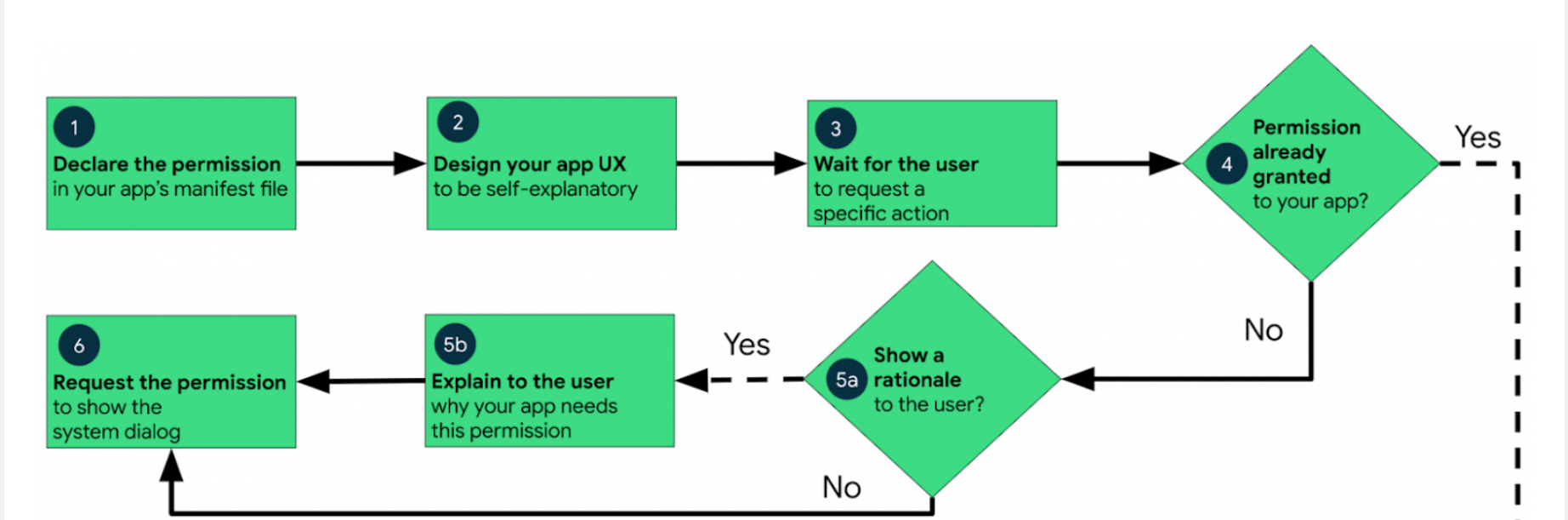
- Регистрировать контракты можно в любой момент жизненного цикла активности или фрагмента, но вот запустить его до перехода в состояние CREATED нельзя. Общепринятый подход — регистрация контрактов как полей класса.
- Не рекомендуется вызывать `registerForActivityResult()` внутри операторов `if` и `when`. Дело в том, что во время ожидания результата процесс приложения может быть уничтожен системой (например, при открытии камеры, которая требовательна к оперативной памяти). И если при восстановлении процесса мы не зарегистрируем контракт заново, результат будет утерян.
- Если запустить неявный интент, а операционная система не сможет найти подходящую Activity, выбрасывается исключение ActivityNotFoundException: “No Activity found to handle Intent”. Чтобы избежать такой ситуации, необходимо перед вызовом `launch()` или в методе `getSynchronousResult()` выполнить проверку `resolveActivity()` с помощью `PackageManager`.

Работа с runtime permissions

Другим полезным применением Activity Result API является запрос разрешений. Теперь вместо вызовов `checkSelfPermission()`, `requestPermissions()` и `onRequestPermissionsResult()`, стало доступно лаконичное и удобное решение — контракты `RequestPermission` и `RequestMultiplePermissions`.

Первый служит для запроса одного разрешения, а второй — сразу нескольких. В колбеке `RequestPermission` возвращает `true`, если доступ получен, и `false` в противном случае. `RequestMultiplePermissions` вернёт `Map`, где ключ — это название запрошенного разрешения, а значение — результат запроса.

В реальной жизни запрос разрешений выглядит несколько сложнее. В гайдлайнах Google мы видим следующую диаграмму:


1 Declare the permission in your app's manifest file
2 Design your app UX to be self-explanatory
3 Wait for the user to request a specific action
4 Permission already granted to your app?
Yes
No
5a Show a rationale to the user?
Yes
No
5b Explain to the user why your app needs this permission
6 Request the permission to show the system dialog
7 Does the user grant permission to your app?
Yes
No
8a Access the info that's protected by the permission
8b Gracefully degrade your app's experience

Зачастую разработчики забывают о следующих нюансах при работе с runtime permissions:

- Если пользователь ранее уже отклонял наш запрос, рекомендуется дополнительно объяснить, зачем приложению понадобилось данное разрешение (пункт 5a)
- При отклонении запроса на разрешение (пункт 8b), стоит не только ограничить функциональность приложения, но и учесть случай, если пользователь поставил галочку “Don't ask again”

Обнаружить эти граничные ситуации можно при помощи вызова метода `shouldShowRequestPermissionRationale()`. Если он возвращает `true` перед запросом разрешения, то стоит рассказать пользователю, как приложение будет использовать разрешение. Если разрешение не выдано и `shouldShowRequestPermissionRationale()` возвращает `false` — была выбрана опция “Don't ask again”, тогда стоит попросить пользователя зайти в настройки и предоставить разрешение вручную.

Реализуем запрос на доступ к камере согласно рассмотренной схеме:

```
class PermissionsActivity : AppCompatActivity(R.layout.a_main) {

   val singlePermission = registerForActivityResult(RequestPermission()) { granted ->
       when {
           granted -> {
               // доступ к камере разрешен, открываем камеру
           }
           !shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
               // доступ к камере запрещен, пользователь поставил галочку Don't ask again.
           }
           else -> {
               // доступ к камере запрещен, пользователь отклонил запрос
           }
       }
   }

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)

       vButtonPermission.setOnClickListener {
           if (shouldShowRequestPermissionRationale(Manifest.permission.CAMERA)) {
               // доступ к камере запрещен, нужно объяснить зачем нам требуется разрешение
           } else {
               singlePermission.launch(Manifest.permission.CAMERA)
           }
       }
   }
}
```

Подводим итоги

Применим знания о новом API на практике и перепишем с их помощью экран из первого примера. В результате мы получим довольно компактный, легко читаемый и масштабируемый код:

```
class NewActivity : AppCompatActivity(R.layout.a_main) {

   val permission = registerForActivityResult(RequestPermission()) { granted ->
       when {
           granted -> {
               camera.launch() // доступ к камере разрешен, открываем камеру
           }
           !shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
               // доступ к камере запрещен, пользователь поставил галочку Don't ask again.
           }
           else -> {
               // доступ к камере запрещен
           }
       }
   }

   val camera = registerForActivityResult(TakePicturePreview()) { bitmap ->
       // используем bitmap
   }

   val custom = registerForActivityResult(MySecondActivityContract()) { result ->
       // используем result
   }

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)

       vButtonCamera.setOnClickListener {
           if (shouldShowRequestPermissionRationale(Manifest.permission.CAMERA)) {
               // объясняем пользователю, почему нам необходимо данное разрешение
           } else {
               permission.launch(Manifest.permission.CAMERA)
           }
       }

       vButtonSecondActivity.setOnClickListener {
           custom.launch("What is the answer?")
       }
   }
}
```

Мы увидели недостатки обмена данными через onActivityResult(), узнали о преимуществах Activity Result API и научились использовать его на практике.

Новый API полностью стабилен, в то время как привычные `onRequestPermissionsResult()`, `onActivityResult()` и `startActivityForResult()` стали Deprecated. Самое время вносить изменения в свои проекты!

Демо-приложение с различными примерами использования Activty Result API, в том числе работу с runtime permissions, можно найти в моем Github репозитории.

Теги: android, android development, android sdk, activity, activitylifecyclecallbacks

Хабы: Блог компании e-legion, Разработка мобильных приложений, Разработка под Android

+12 · 65 · 5

Редакторский дайджест
Присылаем лучшие статьи раз в месяц
Электропочта

e-legion
Делаем приложения, которыми пользуются миллионы
Сайт · Facebook · Twitter · ВКонтакте

6 Карма · 0 Рейтинг
Леонид Беляков @LeonidBelyakov
Android Developer
Подписаться

Комментарии 5

goodvin1709 8 мар 2021 в 23:04

По прежнему большая часть работы остается в Activity, хотелось бы большей энкапсуляции в отдельные классы, что бы этот класс делал все внутренние проверки:
if (shouldShowRequestPermissionRationale(Manifest.permission.CAMERA)) {
// доступ к камере запрещен, нужно объяснить зачем нам требуется разрешение
} else {
singlePermission.launch(Manifest.permission.CAMERA)
}

Возможно это идея для новой библиотеки.

0 · Ответить

CoolMind 14 мар 2021 в 19:42

И в результате разработчики Android значительно усложнили поведение. Принцип SRP здесь совершенно ни при чём, т.к. активность сама по себе часто не SRP.
Проблема в том, что активности могут многократно передавать данные между другими активностями и фрагментами. Длительная навигация по активностям и фрагментам приводит к тому, что приходится постоянно везде передавать setResult(RESULT_OK, data), не менять случайно данные. Если где-то забыли, что прилетит RESULT_CANCELED.
Затем нужно прописать в каждой активности множество коллбеков от других активностей и фрагментов. Затем нужно отследить, от какого фрагмента и от какой активности пришёл коллбек, нигде никакой не забыть. Всё было просто, когда был единый onActivityResult.
Затем надо не забыть вовремя зарегистрировать коллбеки (в onAttach, onCreate) до их первого использования.
В итоге имеем десятки часов отладки в огромном приложении и раздутый код.

0 · Ответить

audiserg 7 мая 2021 в 08:06

Совершенно с вами согласен. Причем ощущение что само SDK идет в другую сторону от каких либо принципов. Но конечно весело шагать в том же направлении и махать флажками гугла, но как то не хочется...

0 · Ответить

CoolMind 7 мая 2021 в 08:38

Спасибо! Мне приятно, что на «Хабре» есть единомышленники. Да, этот ActivityResultApi, с одной стороны, немного упорядочил код, но с другой стороны, сильно его увеличил, а также привёл к многочисленным крэшам. Сколько на нём в результате было поймано багов при сложной навигации (особенно через push-уведомления и DeepLink'и).
Также ежемесячные мучения по переписыванию депрекейтов в андроидных библиотеках.
Меняют не только классы, но целые библиотеки.

0 · Ответить

Vaik_lol 29 июн 2021 в 15:57

они сами то хоть пользовались этой шнягой?

0 · Ответить

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации
ЛУЧШИЕ ЗА СУТКИ · ПОХОЖИЕ

JustJeremy 6 часов назад
Если покупка — это не владение, то пиратство — не воровство
6 мин · 6.9K · Мнение · Перевод
+72 · 35 · 47 +47

Meklon 12 часов назад
Как я собрал красивое ведро для гидропоники
Простой · 10 мин · 7.4K · Туториал
+72 · 77 · 101 +101

dlinyj 9 часов назад
Тюнинг старого тетриса
Простой · 4 мин · 3.8K · Туториал
+43 · 16 · 35 +35

BabayMazay 6 часов назад
Несколько самодельных стеклодувных приспособлений и инструментов. Текущее дооснащение домашней мастерской
Простой · 8 мин · 891 · Туториал
+36 · 9 · 5 +5

slavanikolsky 20 часов назад
Сделано в СССР. Советский истребитель спутников — первым в мире перехватил космический аппарат-мишень
Простой · 10 мин · 21K
+27 · 21 · 69 +69

it_union 11 часов назад
Увольнение без компенсаций: как этого избежать и что вам положено по закону
9 мин · 4.9K
+22 · 27 · 3 +3

Maksvelis 5 часов назад
Выжимаем максимум из PostgreSQL
Средний · 14 мин · 1.9K · Обзор
+21 · 41 · 0

olegantipovDDG 9 часов назад
Роботы-покорители Венеры: 40 лет забытому подвигу советской науки
Простой · 8 мин · 2.1K
+21 · 11 · 6 +6

Antxak 8 часов назад
Мошеннические схемы, так ли их много?
Простой · 5 мин · 1.7K · Мнение
+17 · 14 · 4 +4

ancotir 6 часов назад
Палочки Непера и «Шпаргалка» — карманные «калькуляторы» XVII века для умножения и деления сложных чисел
Простой · 10 мин · 757 · Ретроспектива
+16 · 10 · 1 +1

Как исправить раздвоение встреч в конференциях на базе Jitsi: опыт команды Телемоста
Интересно

Показать еще

МИНУТОЧКУ ВНИМАНИЯ
Турбо — Разбор в деталях: как устроено шардирование Яндекс Диска
Турбо — Как бессонница в час ночной меняет промокодище облик твой
Интересно — Глупым вопросам и ошибкам — быть! IT-менторство на ХК

ВОПРОСЫ И ОТВЕТЫ
Бесконечная установка приложения apk?
Java · Простой · 2 ответа

Как добавить кнопку в контекстное меню в chrome на android?

Android · Простой · 1 ответ

Возможно ли установить приложение в системную папку через adb root?

Android · Средний · 0 ответов

Апп для заучивания иностранных слов по задаваемому словарю?

Linux · Простой · 2 ответа

Нужно опенсорс приложение для Андройд для ESP32 painless mash?

Android · Простой · 0 ответов

Больше вопросов на Хабр Q&A

© 2006–2024, Habr